

Applications of Transformer Networks in Bio- and Cheminformatics

Worksheet 2

Submission Deadline:	28. April 2025, 23:59
Discussion of solutions:	29. April 2024, 12:30 - 14:30

Submission Instructions:

- Create a single Jupyter notebook for all exercises. This Jupyter notebook should contain your code and results. Each submission must include an answer to Exercise 2.4.
- Submit your solutions by uploading the Jupyter notebook to <https://uni-duesseldorf.sciebo.de/s/hCt1rTP23EeWmUC>. The uploaded file should have the following filename "lastname_studentID_worksheet2".

Exercise 2.1 *Separating positional and token type information* (40 Points)

To encode the input sequence, Transformer Networks divide the input sequence into tokens. For each token, we combine information about the token type and its position by simply adding a token embedding and a position vector. Does this lead to a loss of information, or can we still get both information (token type and position) from the resulting vector?

To answer this question, I generated a dataset with 2 500 input embeddings of dimension 1 280 using the trained protein Transformer Network ESM-1b. Each embedding encodes an amino acid and its position in the input sequence. The input embeddings are stored in the file *data/worksheet2/embeddings.npy*, the encoded positions for all embeddings are stored in *data/worksheet2/labels_pos.npy*, and the encoded amino acids are stored in *data/worksheet2/labels_AA.npy* (represented by numbers from 0 to 19).

- (a) Fit a classification model, for example a logistic regression model, to predict the amino acid type from the 1280-dimensional embeddings. First, randomly shuffle the data set and split it into 80% training and 20% test data. Calculate the accuracy on the test set.
- (b) Fit a classification model, for example a logistic regression model, to predict the encoded position from the 1280-dimensional embeddings. First, randomly shuffle the data set and split it into 80% training and 20% test data. Calculate the accuracy on the test set.

Exercise 2.2 *Using HPC*

(30 Points)

For this exercise, follow the instructions on the “1.2 Using_HPC.pdf” slides. Submit all Python scripts and Bash scripts required to perform the following tasks, and any log files resulting from performing the tasks on the HPC.

- (a) Submit a bash script that installs torch (version 2.0.0) for your Python version 3.10.5 on the HPC.
- (b) Submit a bash script that runs a Python script that imports torch and prints out its version.
- (c) Check if torch can find a GPU: Change the requirements in your bash script from `#PBS -l select=1:ncpus=1:mem=8GB` to `#PBS -l select=1:ncpus=1:ngpus=1:mem=8GB`, and add the command `module load CUDA/11.7.1` right after `module load Python/3.10.5` in your bash script. Using this bash script, run a Python script that checks whether a GPU is available.

Exercise 2.3 *Visualizing attention scores*

(30 Points)

The file `data/worksheet2/attention_head_max_P00644.npy` contains the attention scores for a trained 33-layer protein transformer network (ESM-1b) for the protein P00644. For each of the 33 layers, there is a matrix showing the pairwise attention scores for all amino acids in the P00644 protein sequence. Each row of the attention matrix corresponds to the attention scores of one amino acid in the input sequence: That means the row shows the attention scores compared to all other tokens in the sequence that are required to update the amino acid representation.

- (a) Plot all 33 attention matrices by representing the entries of the matrices through colors (similar as in this blog post <https://deepfrench.gitlab.io/deep-learning-project/#Visualizing-the-Attention>).
- (b) Do you see any interesting patterns? How do you interpret them?

Exercise 2.4 *Use of LLMs*

(0 Points)

State for which exercise you have used LLMs (large language models) such as ChatGPT or GitHub Copilot. State which tools you have used and for which steps. This answer does not influence how many points you receive for your submission.